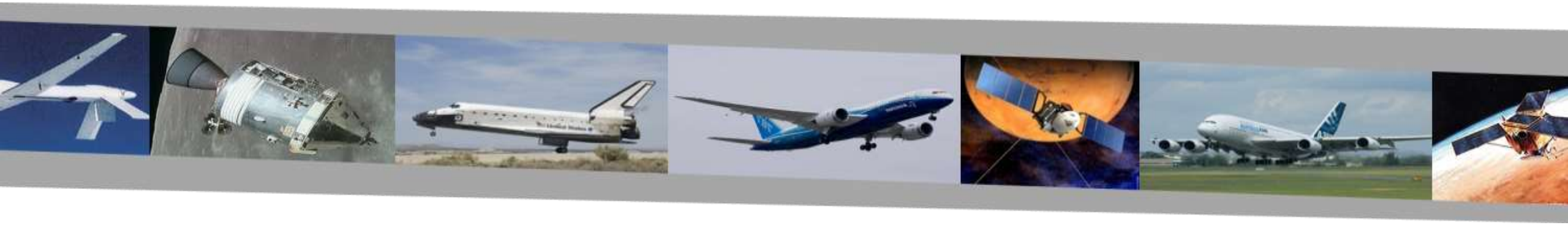
Knowledge Based Engineering (KBE)
AE4202

Dr.ir. G. La Rocca
FPP

Tutorial 4 – Para*Py* geometry (continued)

- *Boolean operations*
- *Topology*
- *Positioning by conditionals*



Content

1. Boolean operations → `exe_16_booleans.py`

   Reference document: **The (usual) ParaPy Tutorials booklet, Exercise 16**

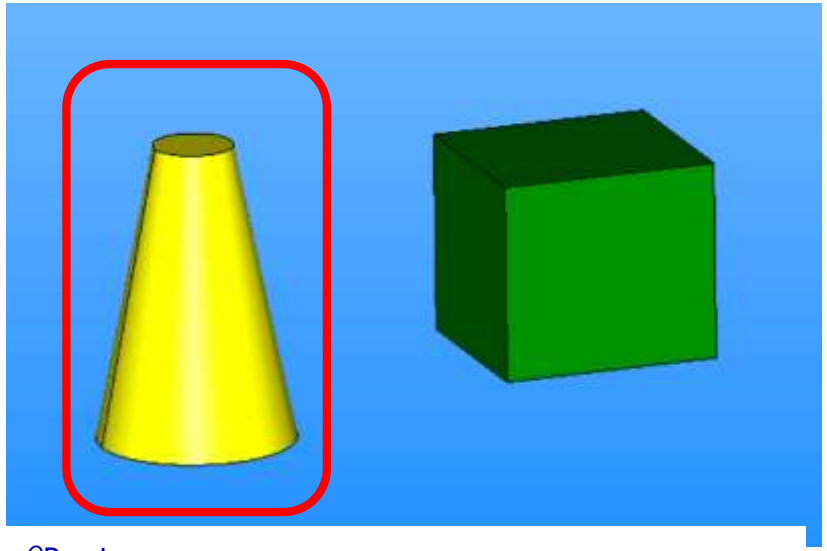
2. Notes on topology → `topology.py`

3. Positioning objects using conditionals → `exe_9_conditional_expressions.py`
   Reference document: **Tutorial 4-Exe 9 on positioning**

Exe 16 on Boolean operations

Boolean operations allows generating complex geometries by uniting, intersecting, subtracting, etc. a set of starting primitives.

In this exercise, your primitives are the ParaPy **`Box`** and **`Cone`**



```
@Part
def cone(self):
    return Cone(radius1=0.5,
                radius2=0.2,
                height=1.5,
                angle=math.pi * 3/2)
```

Note that a `Cone*` instance is defined by 2 radii, 1 height and 1 angle

`radius1` must be not null, but can be smaller than `radius2`

`radius2` defaults to 0 and must be different than `radius1` (you do not get a cylinder when they are equal…)

`height` is the distance between the 2 bases (or the basis and the apex)

Check the use of angle to generate cone portions

Exe 16 on Boolean operations

CTRL+left mouse click on `Cone` or `Box` (or any primitive) in Pycharm to access their source code.

Note the definition of the input for the given class by means of `__initargs__`

```
class Box(Solid):

    __initargs__ = "width, length, height"
```

These arguments need no keyword, so you can create an instance simply like this:

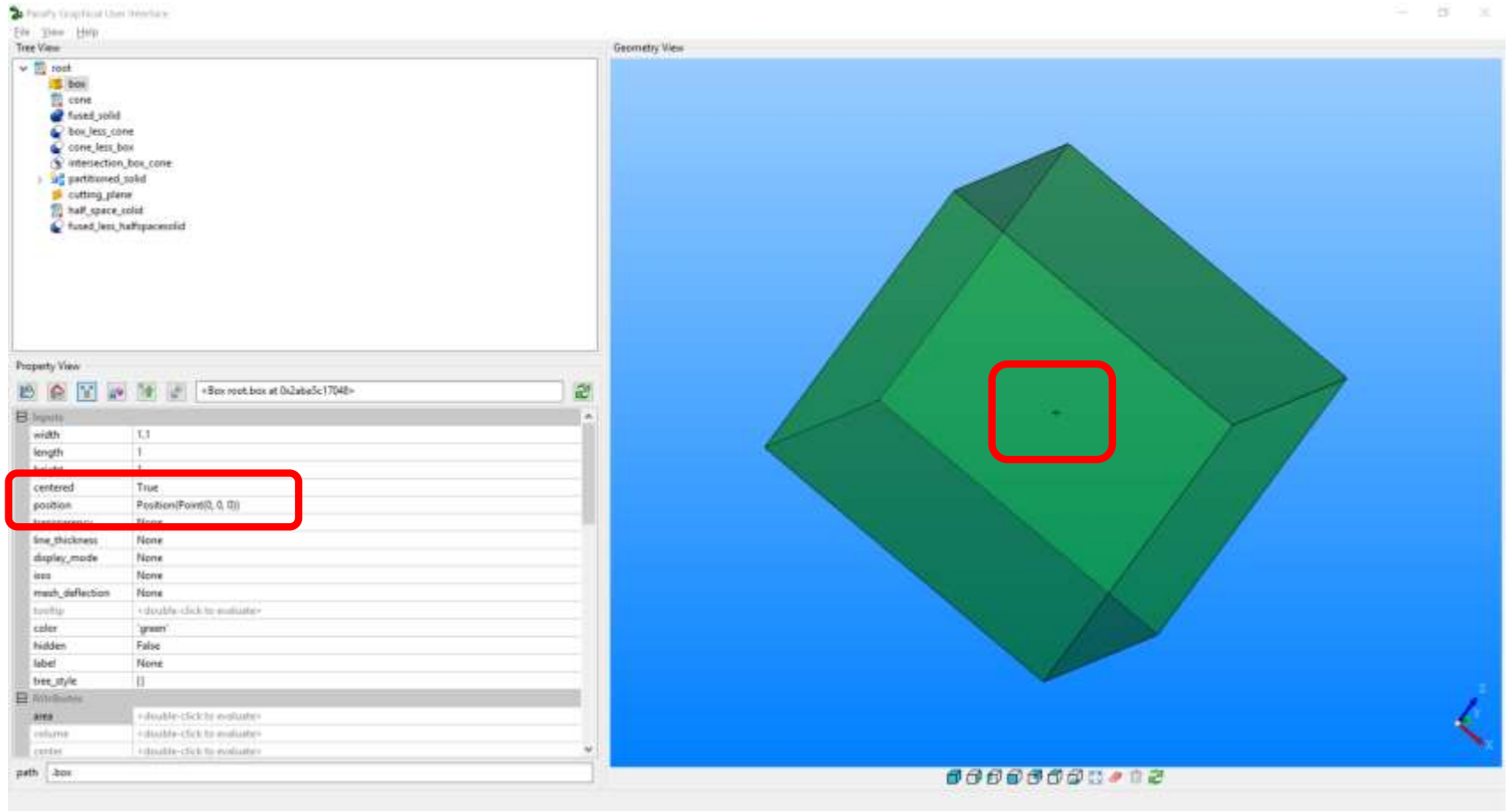
```
@Part
def box(self):
    return Box(1,1,1.1)
```

The first input value will be automatically assigned to `width`, the second to `length`, etc.

You can you can of course still be explicit on the argument names:

```
@Part
def box(self):
    return Box(height=1, length=1, width=1.1)
```

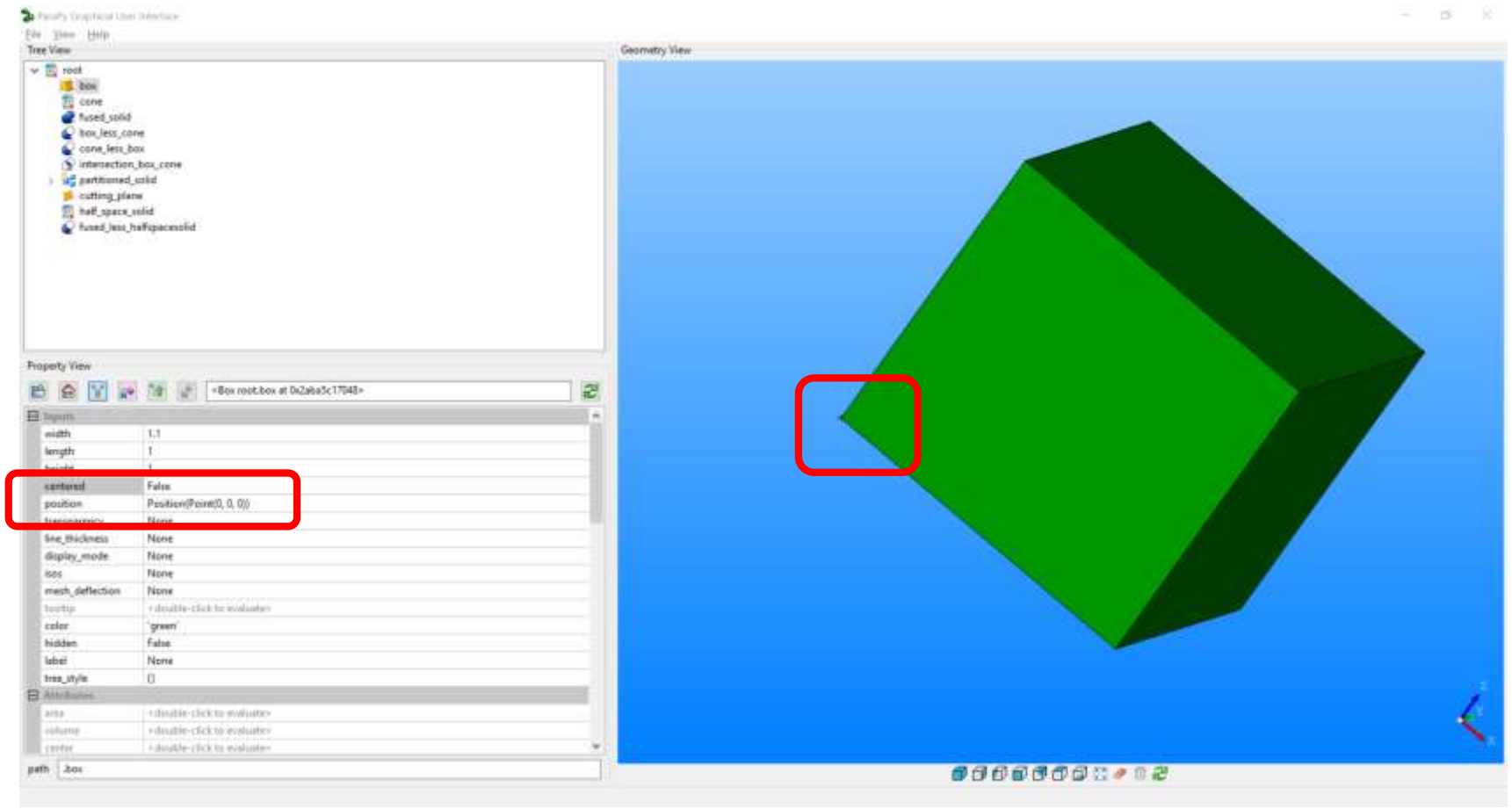
Exe 16 on Boolean operations




```
@Part
def box(self):
    return Box(height=1, length=1, width=1.1,centered=True, color="green")
```

Check the effect `centered` on the positioning of the primitive

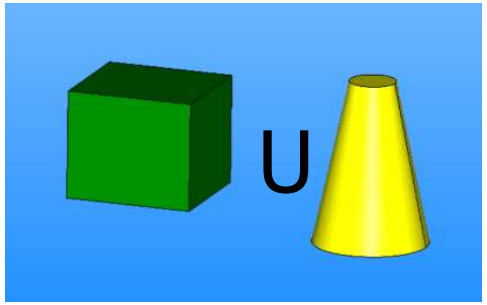
Exe 16 on Boolean operations




```
@Part
def box(self):
    return Box(height=1, length=1, width=1.1,centered=False, color="green")
```

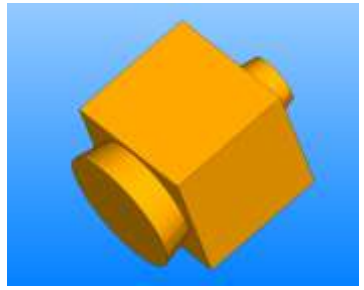
Check the effect `centered` on the positioning of the primitive

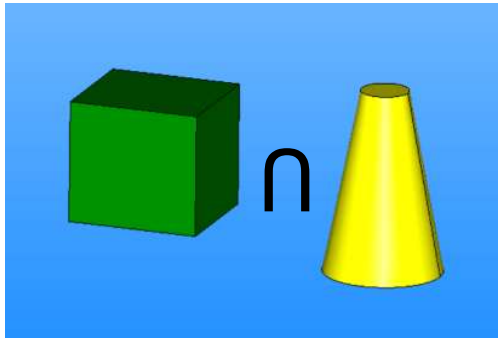
Exe 16 on Boolean operations



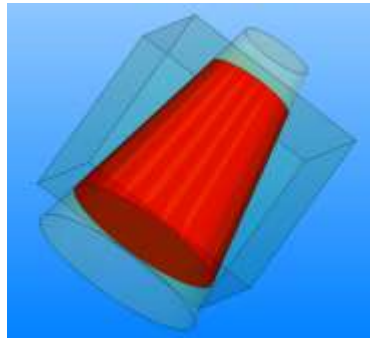
Booleans operations in ParaPy require a `shape_in` and a `tool*`

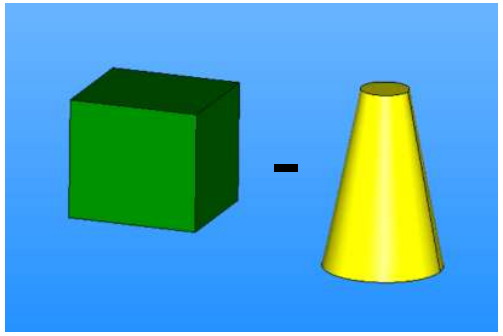
```
@Part
def fused_solid(self):
    return FusedSolid(shape_in=self.box, tool=self.cone)
```



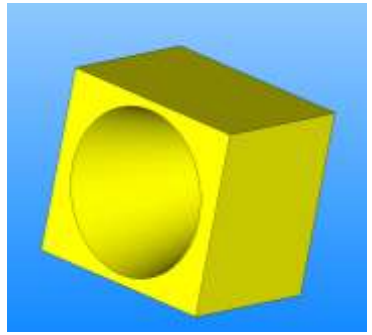



```
@Part
def intersection_box_cone(self):
    return CommonSolid(shape_in=self.box, tool=self.cone)
```

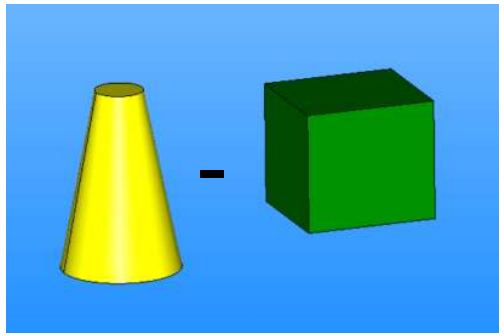




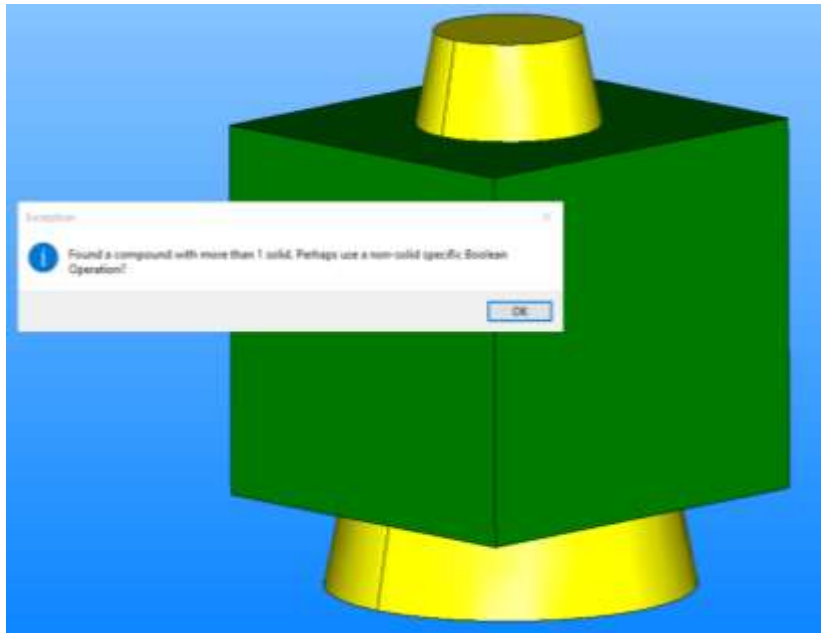

```
@Part
def box_less_cone(self):
    return SubtractedSolid(shape_in=self.box, tool=self.cone)
```



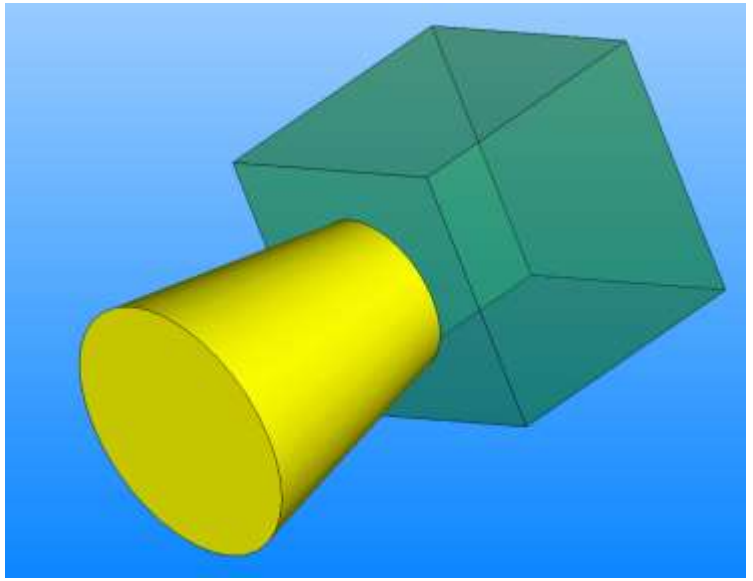
Exe 16 on Boolean operations



```
@Part
def cone_less_box(self):
    return SubtractedSolid(shape_in=self.cone, tool=self.box)
```



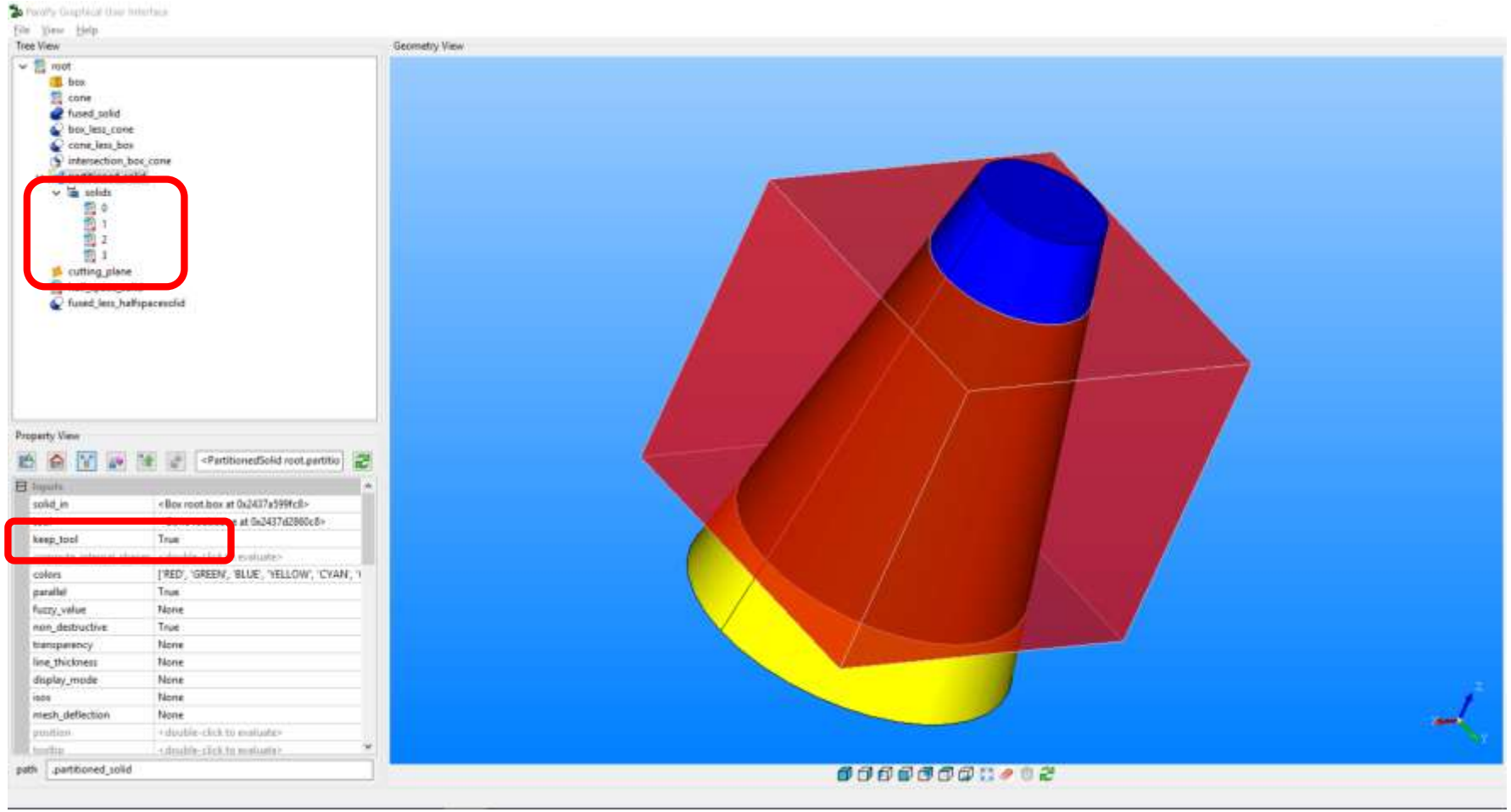
Operation fails when impossible to obtain one single solid



Reposition `cone` such not to pierce `box` and check the outcome

Exe 16 on Boolean operations

```
@Part
def partitioned_solid(self):
    return PartitionedSolid(solid_in=self.box,
                            tool=self.cone,
                            keep_tool=True)
```

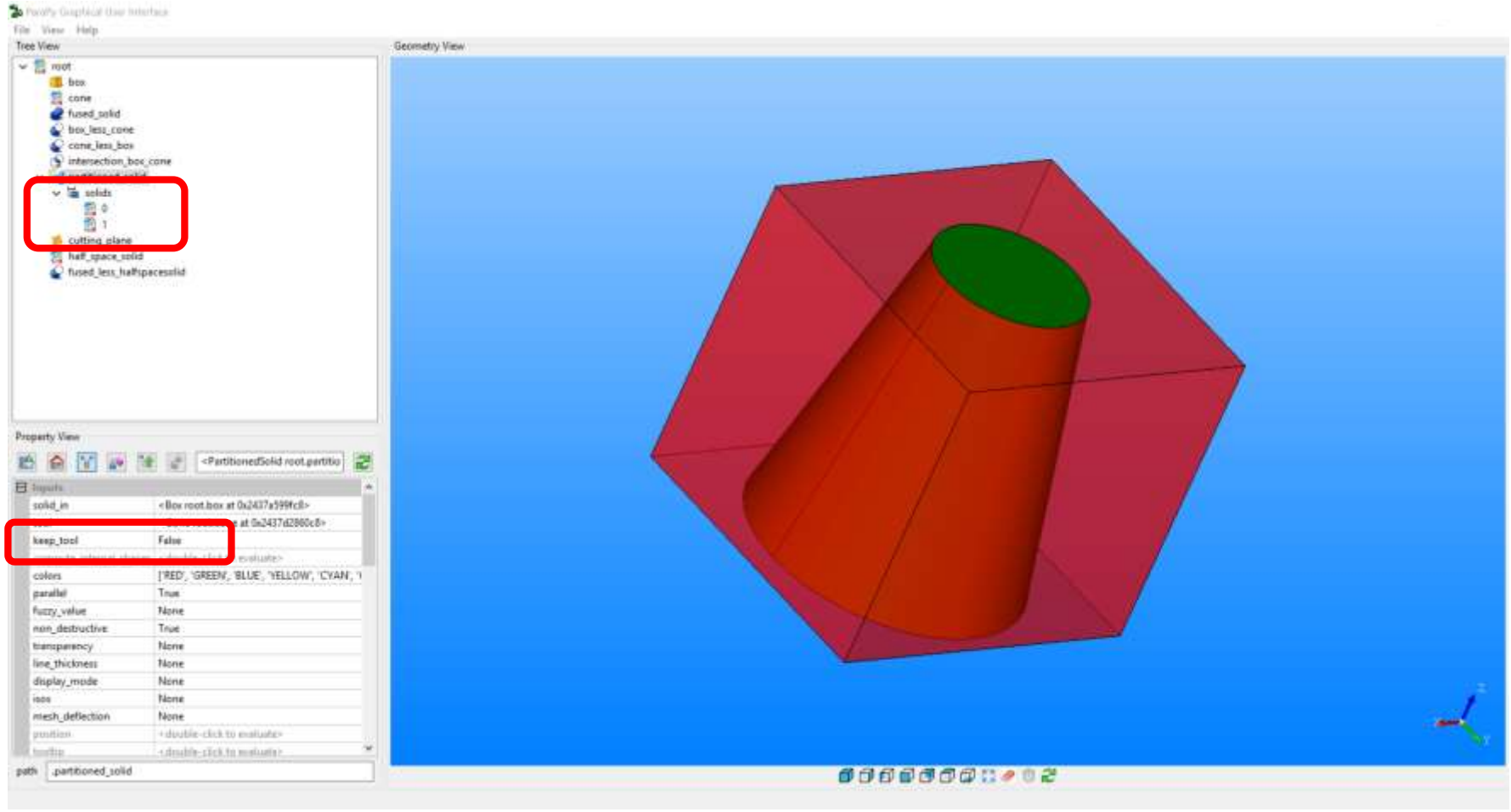


It generates a **sequence** of partitions, **including** those occurring in the `tool`

ATT! Note that this Boolean requires a `solid_in` (rather than a `shape_in`)

Exe 16 on Boolean operations

```
@Part
def partitioned_solid(self):
    return PartitionedSolid(solid_in=self.box,
                            tool=self.cone,
                            keep_tool=False)
```



It generates a sequence of partitions (only those contained in the `solid_in`)

Exe 16 on Boolean operations

`Plane` is a ParaPy class to define infinite* planar surfaces.

It is useful, for example, to cut other geometry entities

CTRL+right mouse click, as usual, to find out how to define a plane, or check the documentation:

class parapy.geom.occ.surface.**Plane**(**args*, ***kwargs*)

Infinite planar surface. Position.Vz is normal.Usage:

```
>>> from parapy.geom import XOY, rotate90, Point, Vector, Plane
>>> # from point and normal vector
>>> pln = Plane(reference=Point(0, 0, 0), normal=Vector(0, 0, 1))
>>> # with explicit binormal ('x')
>>> pln = Plane(reference=Point(0, 0, 0), normal=Vector(0, 0, 1),
...             binormal=Vector(0, 1, 0))
>>> # from Position instance
>>> pln = Plane(rotate90(XOY, 'z'))
```

```
__initargs__ = ["reference", "normal", "binormal"]
```

The most common way to define a `Plane` instance is by providing a `reference` (i.e. **a point**) and the `normal` to the plane (i.e. **a vector**)**

Exe 16 on Boolean operations

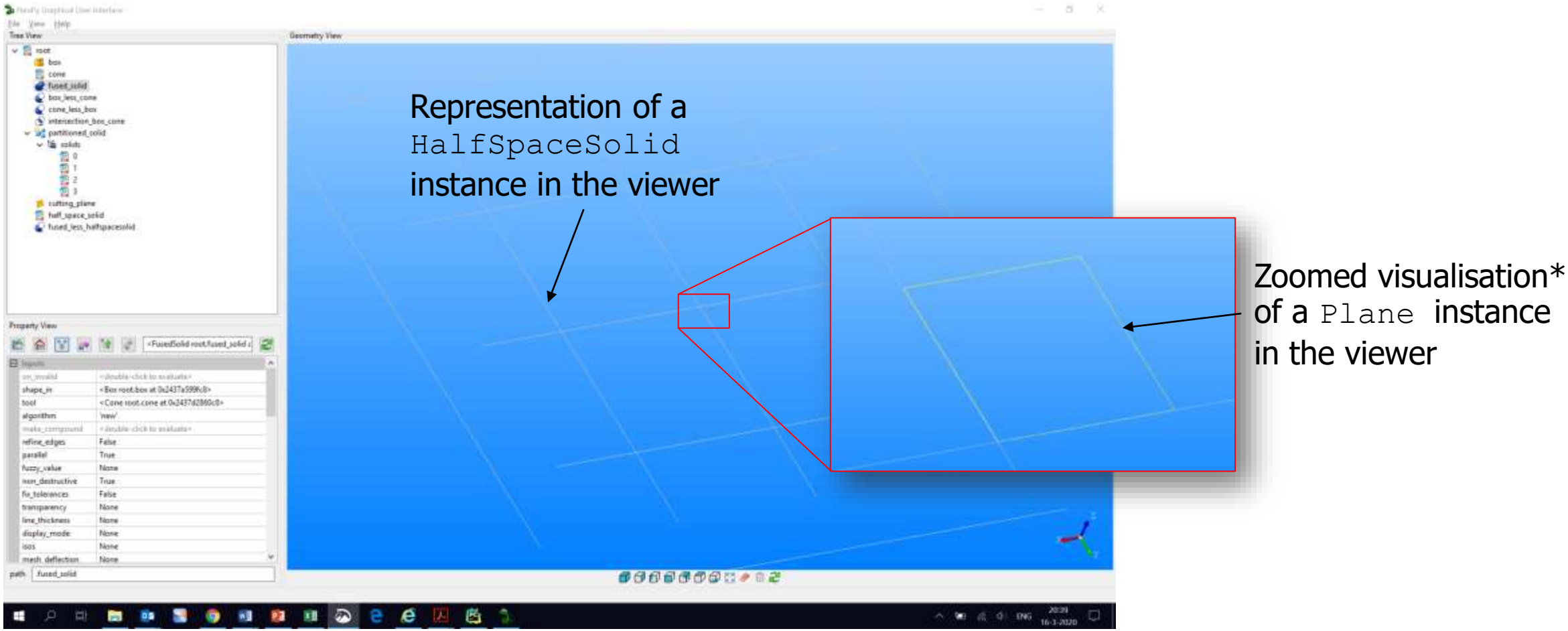
```
@Part
def cutting_plane(self):
    return Plane(reference=self.box.cog, normal=VZ)  # that is how you define a plane


@Part
def half_space_solid(self):
    return HalfSpaceSolid(self.cutting_plane, self.box.cog.translate('z', -0.2))
# the plane above is used to build an infinite solid. In this case: everything below cutting_plane
```

`HalfSpaceSolid` is a class to define an infinite solid, which is filling the space below or above a given surface (e.g. a `Plane` instance).

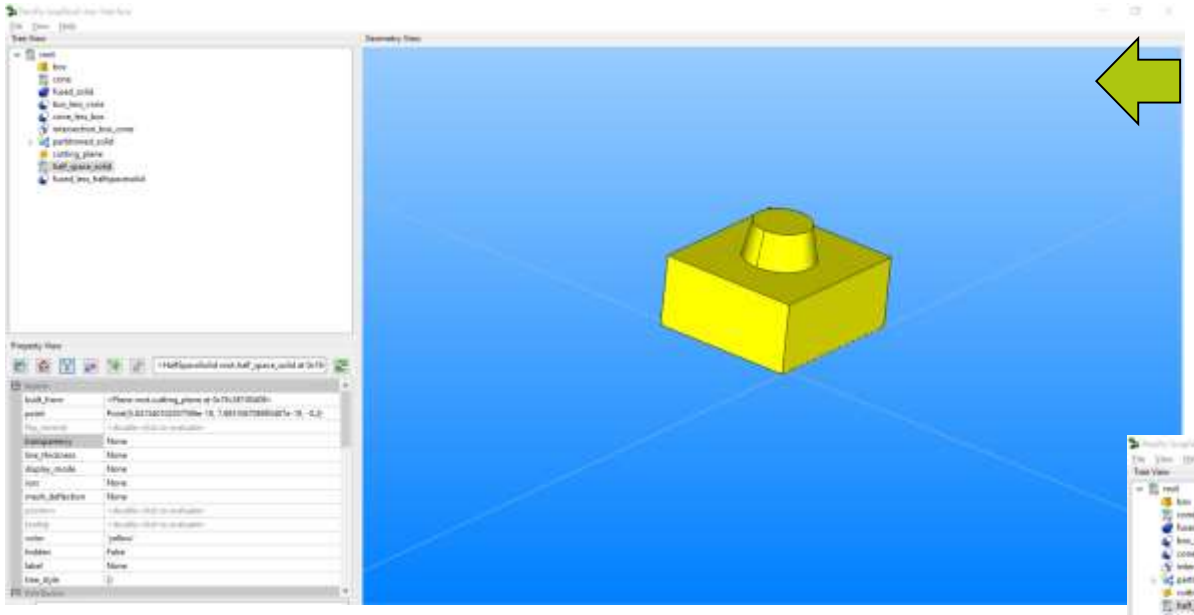
As clear from its `__initargs__ = ["built_from", "point"]` , a surface must be assigned to the input `built_from`, and a point (below or above) to `point`

Exe 16 on Boolean operations

Visualization of a planar instance of `HalfSpaceSolid` and its `build_from` surface (a plane in this case)

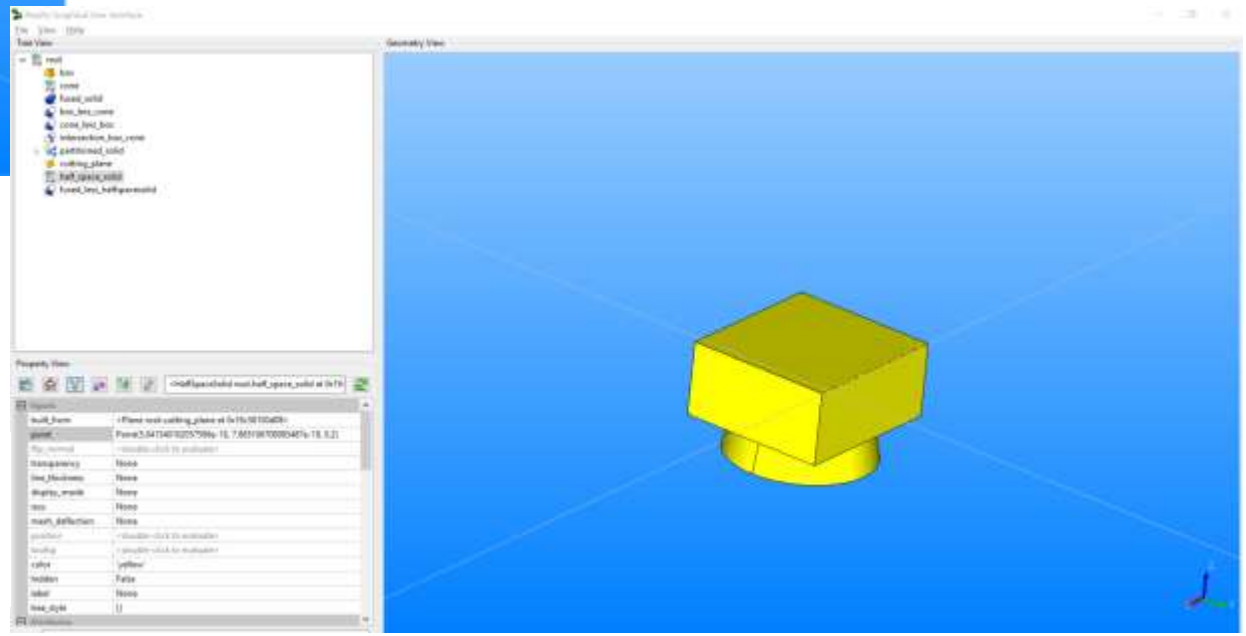


Exe 16 on Boolean operations



```
@Part
def half_space_solid(self):
    return HalfSpaceSolid(self.cutting_plane,
                          self.box.cog.translate('z', -0.2))
```

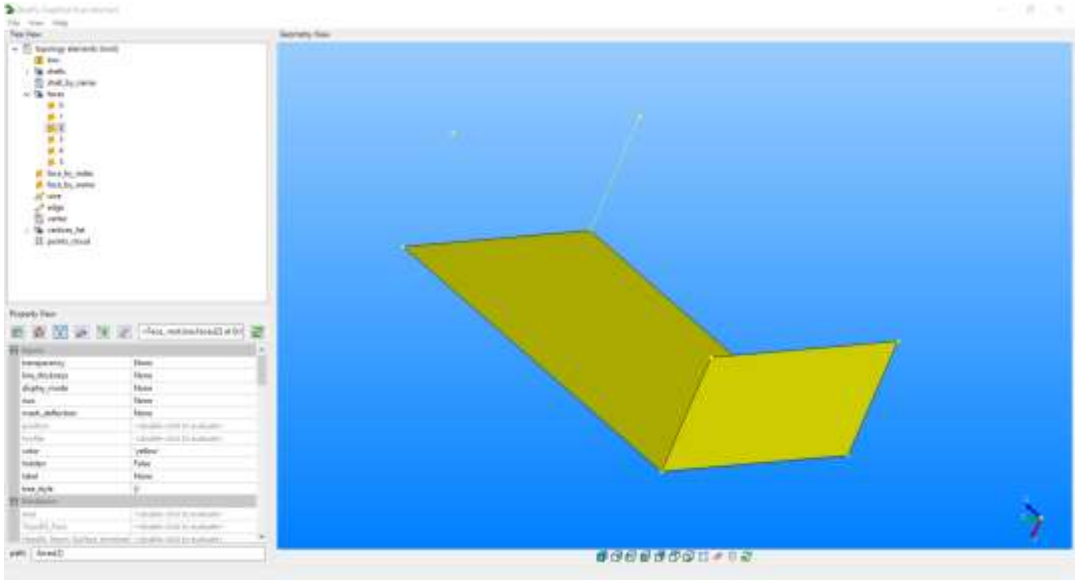
```
@Part
def half_space_solid(self):
    return HalfSpaceSolid(self.cutting_plane,
                          self.box.cog.translate('z', 0.2))
```

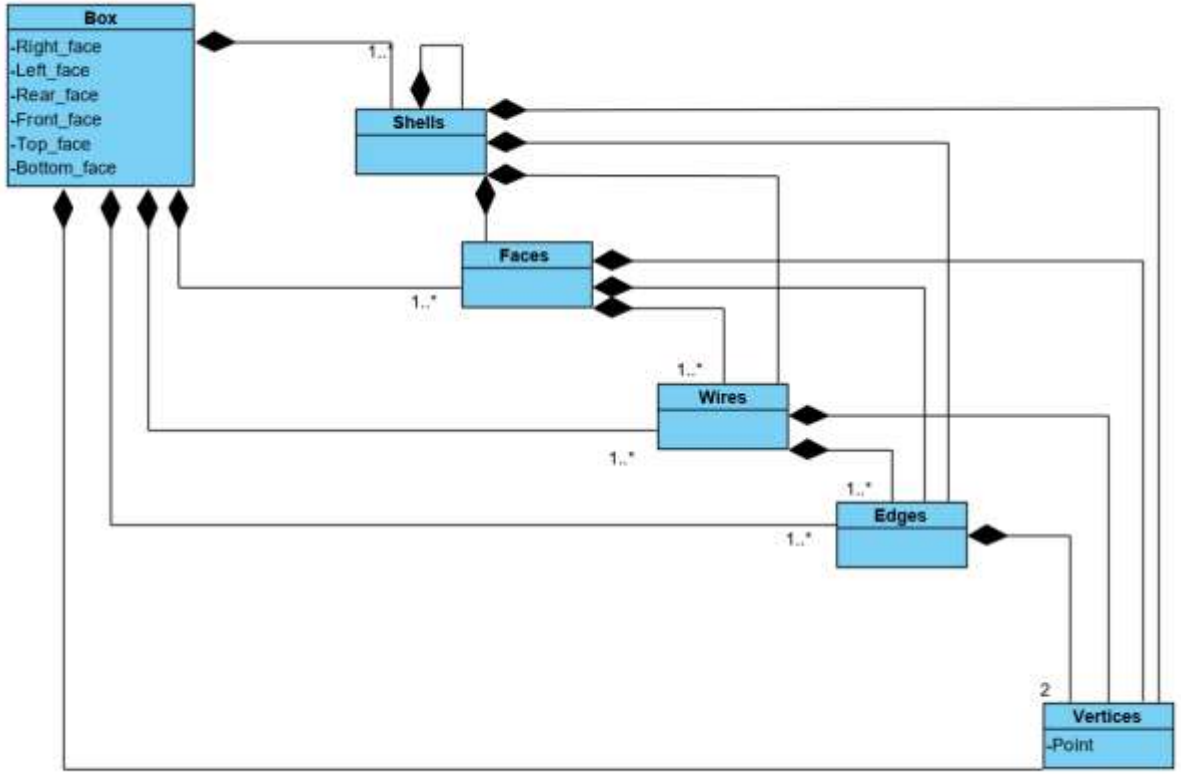


On geometry topology

- To operate at best with geometrical shapes it is convenient to familiarize with their topology. E.g. a Box is build of **Shells**, which include **Faces**, which have **Wires**, etc… (see UML class diagram)
- With the **.dot** notation it is possible to access all topological elements and relative properties, for any geometry primitive

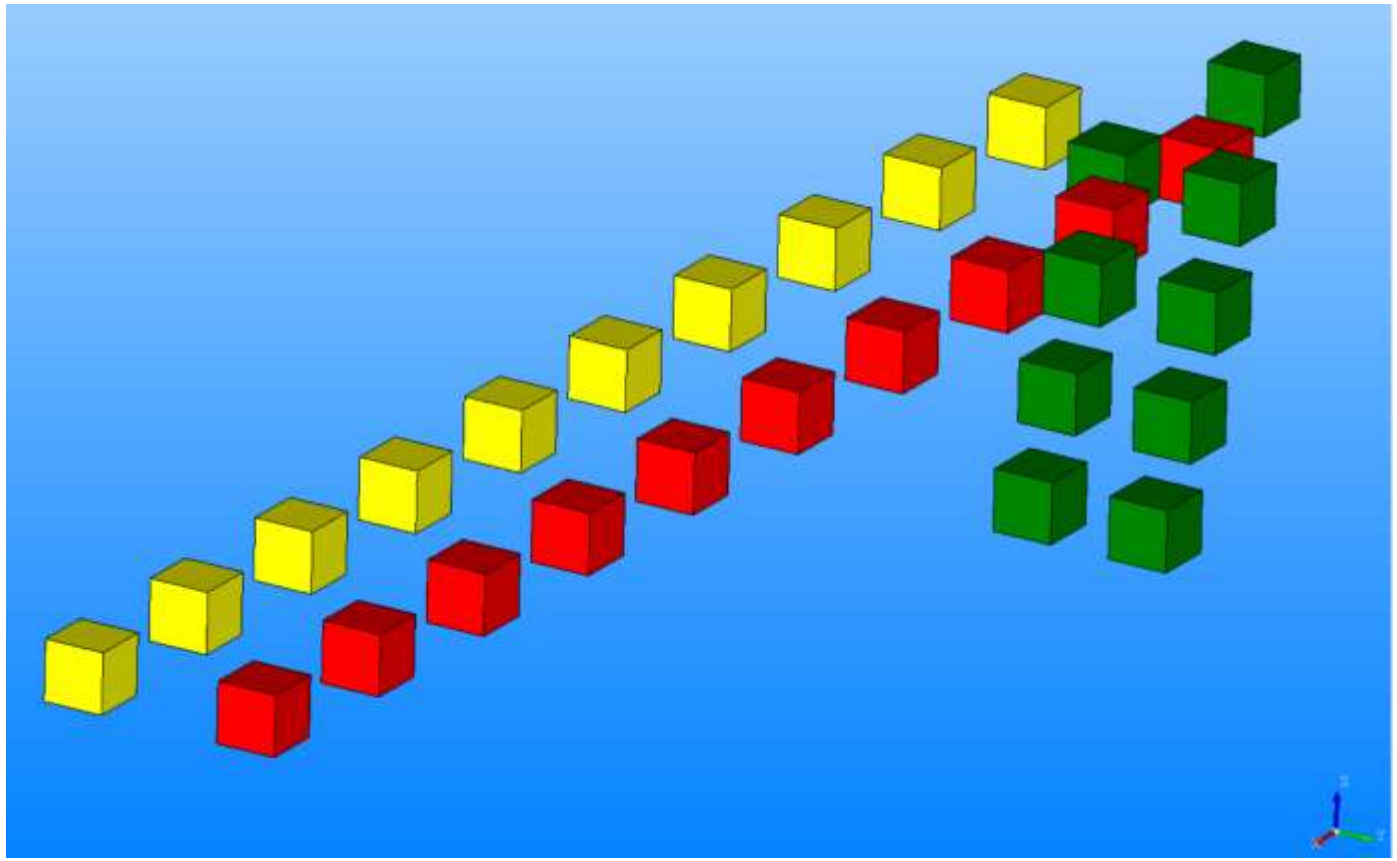
- Use **topology.py** to familiarize with topology concepts





Object positioning with conditionals

exe_9_conditional_expressions.py



Object positioning with conditionals

Assignment: Generate a sequence of 10 boxes. Translate each next box in x-direction with 3 w.r.t. previous

```
@Part
def boxes1(self):
        return Box(quantify=self.n_boxes, height=1, width=1, length=1,
                   position=translate(
                   self.position if child.index == 0 else child.previous.position,
                       'x', 3))
```

The distance to translate

The direction along which to translate (the x axis of the given reference system)

Position used as reference for each translation

In this case it is evaluated using a conditional expression**:

The first `Box` instance* use as reference position the one where the whole sequence `boxes1` is defined***

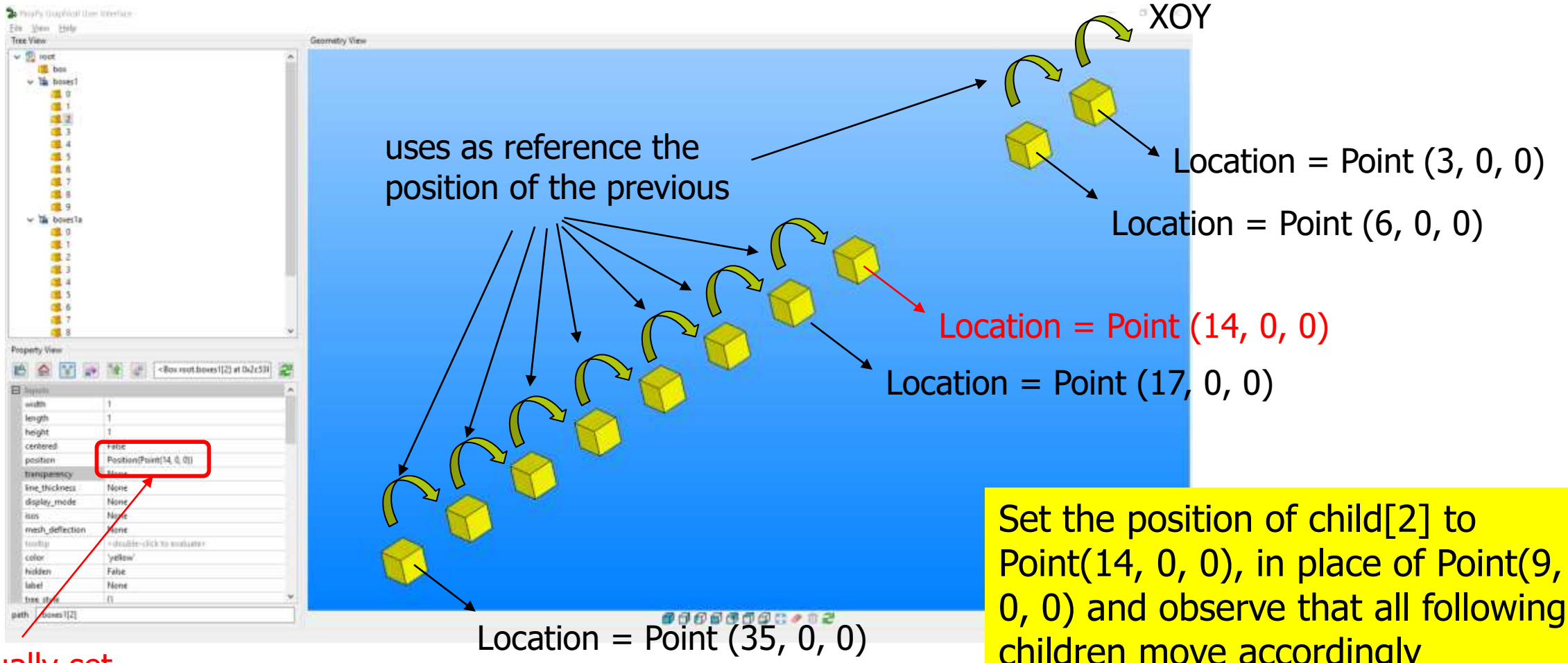
Each following `Box` instance uses as reference the position of the previous instance

→ Each child of the sequence is defined relatively to the previous.

If one child is moved, all the followings move as well (see next slide)

Object positioning with conditionals

Assignment: Generate a sequence of 10 boxes. Translate each next box in x-direction with 3 w.r.t. previous



Set the position of child[2] to Point(14, 0, 0), in place of Point(9, 0, 0) and observe that all following children move accordingly

Conditional syntax

Multi-line conditional syntax:

```
if conditional_1:
     Expression1
elif conditional_2:
     Expression2
else:
     Expression3
```

```
@Part
def boxes1(self):
        return Box(quantify=self.n_boxes, height=1, width=1, length=1,
                   position=translate(
                       self.position if child.index == 0 else child.previous.position,
                       'x', 3))
```

One-line conditional syntax, also called a ternary operator:

```
Expression1 if conditional else Expression2
```

ATT! Arguments definitions in Python only support one-line expressions.
If you are more comfortable with the multi-line syntax, use that to define attributes and use the result in the `@Part` definition.
(This approach also enables more complex rules while keeping code more readable)

Object positioning with conditionals

Variant previous assignment

```
@Part
def boxes1a(self):
    return Box(quantify=self.n_boxes, height=1, width=1, length=1, color="red",
               position=translate(
                   self.position,
                   'x',
                   (child.index + 1) * 3,
                   'y', 3))
```

The direction along which to translate (the x axis of the global reference system)

The distance to translate

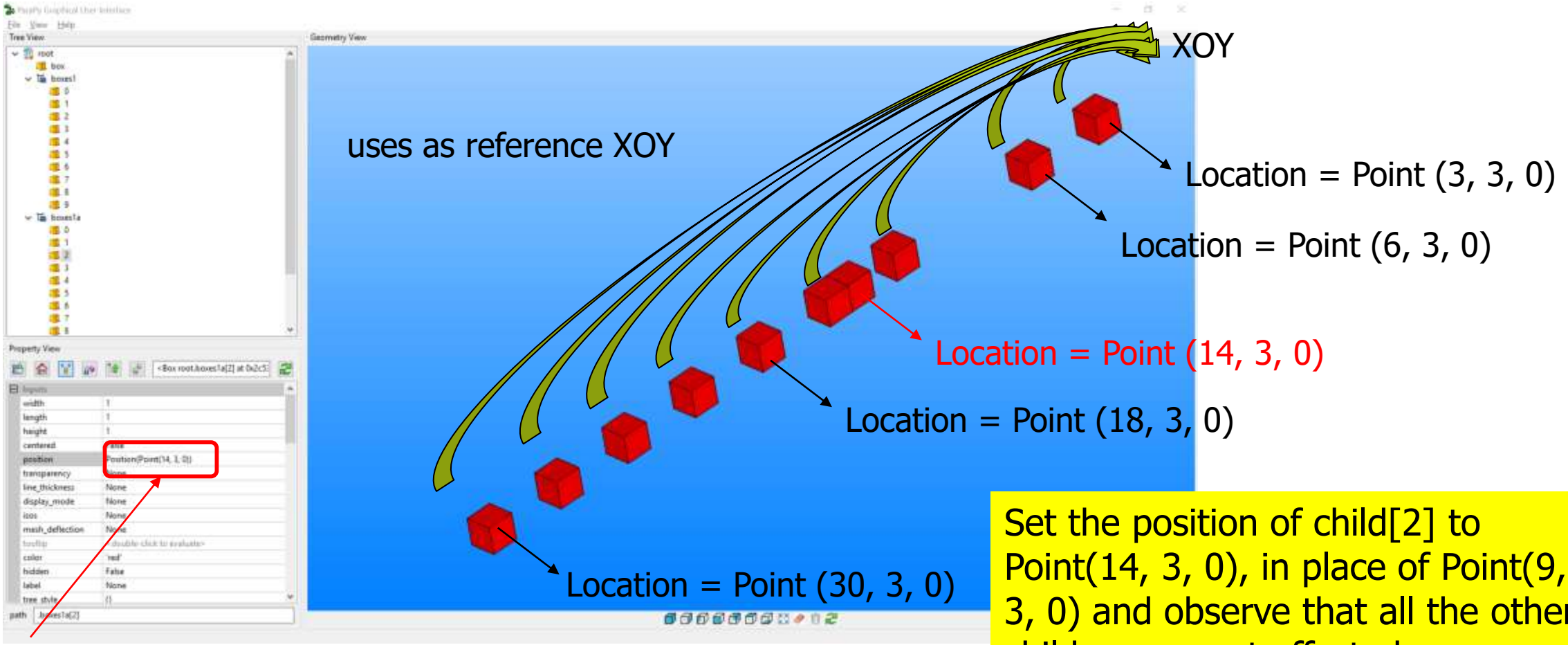
Position used as reference for **all** translations

In this case it is the one where the whole sequence `boxes1a` is defined***

→ Each child of the sequence is defined in the same reference system! (next slide)

Object positioning with conditionals

Assignment: Generate a sequence of 10 boxes. Translate each next box in x-direction with 3 w.r.t. previous



Set the position of child[2] to Point(14, 3, 0), in place of Point(9, 3, 0) and observe that all the other children are not affected

Object positioning with conditionals

Assignment: Anchor first box at `self.position`. Then, translate each next box according to their index:

- even index: translate in 'x' direction by 4
- odd index : translate in 'y' direction by 2

```
@Part
def boxes2(self):
    return Box(quantify=self.n_boxes, width=1, length=1, height=1, color='green',
               position=translate(
                   self.position if child.index == 0 else child.previous.position,
                   'x' if child.index % 2 == 0 else 'y',
                   0 if child.index == 0 else 4 if child.index % 2 == 0 else 2)
               )
```

first child anchored at the location of `self.position`

```
Expression1 if conditional else Expression2
```

Nested one-line conditional syntax, where Expression2 has also one-line conditional syntax

Object positioning with conditionals

Variant on previous assignment to avoid complex one-line conditionals inside the return

```
@Attribute
def positions2(self):
    lst = []  # initialize empty list, we'll fill it with Position instances
    prev = None  # previous Position, initially None
    for i in range(self.n_boxes):
        if i == 0:  # when first index, anchor pos at self.position
            pos = self.position
        else:  # not the first
            if i % 2 == 0:  # even, so translate previous in 'x' by 4.
                pos = translate(prev, 'x', 4)
            else:  # odd, so translate previous in 'y' by 2.
                pos = translate(prev, 'y', 2)
        lst.append(pos)  # add current position to list
        prev = pos  # set previous to current
    return lst  # return our list!

@Part
def boxes2a(self):
    return Box(quantify=self.n_boxes, width=1, length=1, height=1,
               position=self.positions2[child.index])
```

Use an `@Attribute` to evaluate the list of positions and…

…keep the return expression uncluttered